GOLD-001 — Batch 007 Fixes E/F/G, and the Pilot That Could Not Run

Production RAG v1 · 2026-08-30T03:58:20Z · corpus snapshot `snap_689e336380a054d8039dc35b2c09cd0a` · commit `c65d5204901e`

THE FINDING

The calibration pilot did not run. The three preregistered generator defects are implemented and verified against the real candidates that revealed them. The pilot the preregistration requires before the paraphrasing lane may scale could not be run: the frozen evidence it must draw from is not present in this environment, and no substitute for it is admissible. **The paraphrasing lane does not scale, and no batch-007 candidate has been authored.**

3 / 3
preregistered fixes implemented

9/9
whole-sentence labels agreeing with the owner

0 / 10
pilot cases authored — input unobtainable

90
project holdout-eligible, unchanged

1. State verified before anything was written

reads	value	
human_verified	**90**	**PASS**
holdout_eligible	**90**	**PASS**
human_rejected	**9**	**PASS**
genuine multi-hop	1	**PASS**
closure sha256	`7ff5596c755a01fc57cf59b0…`	**recomputed, PASS**
retrieval_was_not_run	`true`	**PASS**
systems_executed	`[]`	**PASS**
SYSTEM-A / SYSTEM-B	`9afcb5b7…` / `304c3509…`	**frozen, unchanged**
batch-007 candidate or pilot artifact	none	**PASS**

The closure hash was **recomputed** from the nine closed batch-006 records rather than read off the closure, so the state is checked rather than quoted.

2. E — cross-library duplicate facts

`src/rag_v1/gold/factidentity.py`

compares the normalised (subject, relation, object) triple in addition to text and offsets; flags, never drops; reports 'not_comparable' when no triple can be read

The pair the owner caught is now visible. `GOLD-B005-11` (OpenAI Python library) and `GOLD-B006-06` (TypeScript/JavaScript library) share no question text, no span offsets and no span text — which is exactly why the old comparison could not see them. Both now normalise to the triple `('aws_bedrock_base_url', 'override', 'endpoint')`, and the second is flagged.

```
B005-11  Pass `base_url` to `bedrock(...)` or set `AWS_BEDROCK_BASE_URL` to override the derived `https://bedrock-mantle.
<region>.api.aws/openai/v1` endpoint.
B006-06  … and `AWS_BEDROCK_BASE_URL` can override the endpoint.
```

Batch 005 predates the triple fields, so its triple is **derived from its frozen evidence** — never from its question, because the evidence is the part that cannot drift. Within batch 006 the check raises **0** false positives. It **flags and never drops**: two libraries genuinely differing in behaviour is a real case, and only a reviewer can tell that apart from a restatement.

3. F — reasoning type read from the whole sentence

`src/rag_v1/gold/reasoningtype.py`

classifies from the whole sentence; lifecycle first whatever the verb; configuration_interaction requires an interaction verb, so one requirement naming two identifiers is a lookup

id	generated	owner	whole-sentence	
01	configuration_interaction	exact_lookup	exact_lookup	**PASS relabelled**
02	error_behavior	error_behavior	error_behavior	**PASS**
03	exact_lookup	lifecycle_compatibility_migration	lifecycle_compatibility_migration	**PASS relabelled**
04	exact_lookup	exact_lookup	exact_lookup	**PASS**
05	exact_lookup	exact_lookup	exact_lookup	**PASS**
06	configuration_interaction	configuration_interaction	configuration_interaction	**PASS**
07	configuration_interaction	configuration_interaction	configuration_interaction	**PASS**
08	configuration_interaction	lifecycle_compatibility_migration	lifecycle_compatibility_migration	**PASS relabelled**
09	exact_lookup	exact_lookup	exact_lookup	**PASS**

9/9 agreement with the owner's labels, including all **3** the owner had to relabel. Lifecycle is tested first, whatever the verb: a compatibility sentence almost always also contains a lookup verb, and reading it as a lookup is precisely how `GOLD-B006-03` was mislabelled.

4. G — a scoped source needs a scoped question

`src/rag_v1/gold/questionscope.py`

a scope qualifier in the evidence must appear in the question AND in the critical strings, else the candidate does not export; a comparative aside is not a scope

id	as generated	as owner-approved
02	`SCOPE_MISSING_FROM_CRITICAL_STRINGS` — **dropped**	`SCOPED` — **exports**
04	`SCOPE_MISSING_FROM_QUESTION` — **dropped**	`SCOPED` — **exports**
05	`SCOPE_MISSING_FROM_QUESTION` — **dropped**	`SCOPED` — **exports**

The second condition is the one with teeth. A qualifier that appears in the question but in no critical string is decoration: nothing downstream reads it. Requiring it in the critical strings is what turns this from a style rule into a gate.

> **A FINDING THE REVIEWER MUST DECIDE — G-STRICTER-THAN-BATCH-006**
>
> The preregistered rule requires the scope qualifier in the question AND in the critical strings. GOLD-B006-08 carries 'OpenAI Python SDK' in its question but not in its critical strings, and the owner approved it. Implemented as preregistered rather than loosened; recorded so the reviewer can decide whether batch 007 should hold to the stricter rule.
>
> *recorded as a passing test, not tuned away.*

> **INDEPENDENT AUTOMATED-REVIEW RECOMMENDATION — NOT PROJECT-OWNER APPROVAL**
>
> A recommendation only. It does not approve, verify or close anything, does not set human_verified, and does not alter the preregistration. Only the project owner may approve, and the preregistration is amended only by the owner.
>
> Recorded 2026-08-30T04:04:55Z · concerns finding G-STRICTER-THAN-BATCH-006

R1. Keep Gate G exactly as preregistered for batch 007.

The preregistered rule requires the scope qualifier in the question AND in the critical strings. The second condition is the one that has effect: a qualifier appearing only in the question is read by no check, which is how two of batch 006's three rescoped candidates passed with the qualifier absent from the critical strings entirely. Loosening the gate to match what batch 006 accepted would remove the part that does the work, and a rule relaxed after seeing which candidates it would drop is a rule fitted to the output — the precise failure preregistration exists to prevent.

R2. Grandfather GOLD-B006-08 under the closed batch-006 standard. Do not rewrite it, do not relabel it, do not re-open it.

GOLD-B006-08 was authored, reviewed and approved by the owner under the rule in force for batch 006, and batch 006 is closed and hash-covered. A closed batch is a historical artifact: the standing constraint is that it is never edited, and an erratum, audit or versioned overlay is the only admissible instrument. A stricter rule preregistered for batch 007 governs batch 007. Applying it backwards would rewrite an approved record to satisfy a rule that did not exist when the owner approved it, and would break the closure hash that covers it.

Consequence accepted. Gate G will drop batch-007 candidates that batch 006 would have accepted. That is the intended effect of the stricter rule, not a regression, and the drop count belongs in the batch-007 generation report so the cost is visible.

What this changes in the code: Nothing. Gate G is already implemented as preregistered. **Status of the finding:** Still open for the project owner. The recommendation is recorded, not applied as a decision.

5. The calibration pilot was not run

> **BLOCKED**
>
> BLOCKED — the frozen evidence the pilot must draw from is not present in this environment

The preregistration fixes the pilot's input exactly: *10 evidence spans that failed batch 006 ONLY because no builder could express them — NO_BUILDER / UNBUILDABLE.* That input cannot be obtained here, on four independent grounds.

1. **the NO_BUILDER/UNBUILDABLE set was counted, never persisted.** batch 006 recorded removed.unbuildable = 2482 as an integer; scripts/export_batch_006.py:733 increments the counter and returns, discarding the fact's identity. No artifact records which spans they were.
2. **the corpus text exists only in Postgres, and this container has no corpus.** load_docs() reads document_version.normalized_text joined to corpus_snapshot_version. The local cluster starts but holds only postgres/template0/template1 — there is no 'rag' database and no snapshot.
3. **the raw documents are not in the repository.** data/raw/ contains only .gitkeep and is gitignored by design; data/cache/ is empty.
4. **re-fetching would not reproduce the frozen snapshot.** data/manifests/v1-openai-anthropic.yaml lists 202 documents captured 2026-08-17 with no text and no content hashes. Re-fetching returns the documentation as it stands today, so offsets and evidence hashes would not match snap_689e336380a054d8039dc35b2c09cd0a, and the evidence would not be the frozen evidence.

No pilot case was authored. Authoring 10 cases against invented or re-fetched evidence would produce exactly what the preregistration exists to prevent — a benchmark testing what its author imagined rather than what the documentation says — and the pilot's four thresholds would measure nothing.

The four thresholds, unmeasured

criterion	threshold	measured
independently judged factually sound	>= 8 of 10	**not measured — pilot not run**
unsupported claims	0	**not measured — pilot not run**
relation direction reversals	0	**not measured — pilot not run**
scope broadening	0	**not measured — pilot not run**

To unblock: Restore the corpus snapshot into Postgres (or restore data/raw/ and re-ingest), then re-run batch 006's miners to re-derive the spans that reach no builder and take the pilot's 10 from that set.

Until the pilot runs and is independently reviewed, the paraphrasing lane does not scale. The preregistration is explicit that a failed pilot means revising the authoring contract and re-piloting — not proceeding — and an *absent* pilot is not a weaker condition than a failed one.

Recovery checklist

The ordered, precise steps that unblock the calibration pilot. Nothing here has been performed; the pilot remains not run.

> **NOT ADMISSIBLE AS A SUBSTITUTE FOR THE FROZEN CORPUS**
>
> - Do NOT substitute the current live contents of the manifest URLs. The documentation has moved on from the capture date, so offsets and evidence hashes would not match the frozen snapshot and the evidence would not be the frozen evidence.
> - Do NOT substitute toy, synthetic or fixture data. tests/fixtures/docs/widget_v2.md is a synthetic document for unit tests and is not corpus.
> - Do NOT reconstruct spans by hand from quotations in closed batch records.
> - Do NOT run any retrieval system at any step; SYSTEM-A and SYSTEM-B stay frozen.

#	step	expected / done when
1	**Restore the exact frozen source-corpus snapshot into the expected PostgreSQL schema.** Create the `rag` database and role and apply sql/001_init.sql, sql/002_chunk_sets.sql, sql/003_search_text.sql and sql/004_embedding_cache.sql, then restore the corpus from a backup of the snapshot — or restore data/raw/ (gitignored, 202 documents) and re-ingest it. The snapshot must be the one captured 2026-08-17T04:46:19Z, not a re-fetch.	*Expected:* document_source, document_version and corpus_snapshot_version populated; snapshot_id snap_689e336380a054d8039dc35b2c09cd0a present. *Done when:* scripts/export_batch_006.py's load_docs() returns 202 documents.

#	step	expected / done when
2	**Verify the restored corpus is the frozen one, by fingerprint, before using it.** Identity is not the snapshot id — that is a label anyone can write. Re-read each closed batch-006 span from the restored corpus at its recorded (version_id, char_start, char_end) and re-hash it; every evidence_hash must reproduce exactly. Batch 006's nine records give nine independent checks against text that cannot have drifted, and batches 001-005 give more. Also confirm rag_v1.systems.FROZEN_HASHES still equals SYSTEM-A 9afcb5b7… and SYSTEM-B 304c3509….	*Expected:* every re-hash matches its recorded evidence_hash; document count 202. *Done when:* zero hash mismatches. A single mismatch means the restored corpus is not the frozen snapshot — stop, do not proceed to step 3.
3	**Deterministically re-derive the batch-006 NO_BUILDER / UNBUILDABLE span set.** The set was counted, never persisted: scripts/export_batch_006.py:733 increments removed['unbuildable'] and returns, discarding the fact. Re-run batch 006's miners against the restored snapshot at the same commit and record, for each fact where the builder returns None, its (version_id, char_start, char_end, evidence_text) rather than only a count. Do not change any builder while doing this: the set is defined by the builders as they were.	*Expected:* the recorded count reproduces exactly: 2482 unbuildable facts. *Done when:* the re-derived count equals 2482. A different count means the derivation is not reproducing batch 006's conditions — diagnose before selecting anything.
4	**Select the preregistered 10 pilot cases from that set, and only from it.** Take 10 spans that failed batch 006 ONLY because no builder could express them. Exclude any span that failed a semantic gate — those failed for reasons paraphrasing does not fix — and exclude any span already spent by a closed batch. Record the selection basis for each so the choice is auditable and not a convenience sample.	*Expected:* 10 spans, each traceable to the step-3 set. *Done when:* each of the 10 carries its version_id, offsets and selection basis.
5	**Run the calibration pilot and measure the four preregistered thresholds.** Author in the preregistered order — frozen evidence, literal source fact, subject/relation/object, atomic claims, and only then the paraphrased question — never the forbidden order. Record all eight required fields on every case. Run the A-G entailment self-check, where any failure is a DROP with no flag-and-continue branch, then every retained gate including the new E, F and G. No retrieval is run on the pilot.	*Expected:* factually sound >= 8 of 10; unsupported claims 0; relation-direction reversals 0; scope broadening 0. Wording cleanup does not count against the criterion. *Done when:* the pilot is independently reviewed against those four thresholds. If it fails, do not scale the lane: revise the authoring contract and re-pilot.

6. Recovery search — every path, and what was in it

NO — EXHAUSTED; THE SNAPSHOT IS NOT PRESENT IN ANY REACHABLE LOCATION. CORROBORATED BY AN INDEPENDENT HOST-SIDE SEARCH (HOST_SIDE_RECOVERY_EVIDENCE), BY THE ASSESSMENT OF THE 2026-08-17 PUBLISHED RESULTS (PUBLISHED_RESULTS_EVIDENCE), AND BY THE CHATGPT-PROJECT ARCHIVE AUDIT (CHATGPT_PROJECT_ARCHIVE_EVIDENCE) WHICH CLOSED THE PRODUCTION-RAG-V1.ZIP LEAD AS EARLY SCAFFOLD CARRYING NO CORPUS.

Can the exact frozen corpus snapshot snap_689e336380a054d8039dc35b2c09cd0a, captured 2026-08-17T04:46:19Z, be recovered non-destructively from anything reachable in this session?

Searched 2026-08-30T10:59:40Z

path searched	method	result
git history — all refs, all commits	git log --all --diff-filter=A over data/raw/*; git rev-list --all with ls-tree for every path ever present; search of every added path for dump/archive/backup extensions	Only data/raw/.gitkeep was ever tracked (added in 185bc3a). No database dump, archive or corpus file has ever been committed on any branch.
tracked repository artifacts	scanned every evals/, experiments/ and docs/ JSON and JSONL for document or chunk body text	The largest carrier, experiments/EXP-011/results.json, holds 9858 characters of query-result snippets across 0 identifiable version ids. Closed records reference offsets up to character 2711869, so the corpus is at least ~2.7M characters: the artifacts hold under 0.4% of it, with no version ids and no offsets into normalized_text. Reconstruction is impossible and would in any case be the prohibited hand-reconstruction.
prior-session artifacts, manifests, logs and reports	searched the whole filesystem for any file naming the snapshot id	The only file outside the repository naming snap_689e336380a054d8039dc35b2c09cd0a is this session's own transcript at /root/.claude/projects/. No prior-session artifact carries corpus text.
database dump or backup locations named by project documentation	grep across all Markdown, Python, YAML and the Makefile for pg_dump, pg_restore, backup, .dump and restore instructions	The project documentation names no backup or dump location anywhere. There is no documented restore path; the corpus was only ever produced by scripts/fetch_corpus.py into gitignored data/raw/ and ingested from there.
Docker volumes (docker-compose.yml declares rag_pgdata)	inspected /var/lib/docker/volumes and the Docker daemon	/var/lib/docker/volumes does not exist and no Docker daemon is running. The rag_pgdata volume has never existed in this container.
PostgreSQL data directories	pg_lsclusters; started the 16/main cluster; enumerated databases, user tables and WAL	One cluster, initdb'd 2026-03-31 at image build. It holds only postgres, template0 and template1, zero user tables, and a single WAL segment 000000010000000000000001 — it has never held project data. A second finding: the pgvector extension is not available in this cluster at all (0 rows in pg_available_extensions, no extension files on disk), so the expected schema cannot even be created here, let alone populated, because sql/001_init.sql requires the VECTOR type.
persistent Claude workspace and mounted volumes	findmnt for non-overlay mounts; listed /home/user, /mnt/user-data, /mnt/attach, /srv, /root/.claude/backups and /root/.claude/projects	All empty of corpus. /mnt/user-data/working, /mnt/attach and /srv are empty directories; /root/.claude/backups holds only Claude configuration backups.
filesystem-wide archive sweep	find / -xdev for *.dump, *.pgdump, *.sql.gz, *.tar, *.tar.gz, *.tgz, *.bak and *backup* over 100KB, excluding system and package directories	No candidate found. No provider documentation exists anywhere outside the repository.

Constraints observed throughout.

- No current live URL was fetched.
- No toy or fixture data was used or accepted as corpus.
- SYSTEM-A and SYSTEM-B were not run.
- Validation and holdout were not inspected.
- Fingerprint proof was required before any corpus would have been accepted; none was offered because none was found.

7. Host-side recovery evidence

> **EXTERNAL AUTOMATED SEARCH EVIDENCE — NOT PROJECT-OWNER APPROVAL**
>
> An independent automated host-side search, reported to this session. It approves nothing, verifies nothing about the benchmark, and does not set human_verified. Only the project owner may approve.
>
> Recorded 2026-08-30T11:06:02Z · read-only searches; nothing on the host was modified

Provenance. Performed on the owner's host machine and relayed to this session as a result. It was NOT performed or independently confirmed from inside this container: this session cannot see the host filesystem, so these findings are recorded as reported, not as verified. They are consistent with, and extend to the host, the in-session search recorded under recovery_search.

Locations searched. `C:\Users\yorkr\Documents`, `C:\Users\yorkr\Downloads`, `OneDrive`, `projects`, `Claude`, `.claude`, `nyxora-recovery`

Looked for:

- a RAG corpus dump
- a PostgreSQL backup
- a byte-identical raw capture
- the snapshot ID snap_689e336380a054d8039dc35b2c09cd0a
- a corpus_snapshot_version artifact

area	result
user document, download, OneDrive, project and Claude directories	No RAG corpus dump, PostgreSQL backup, byte-identical raw capture, snapshot ID or corpus_snapshot_version artifact was found outside the repository.
Claude local application data	No matching persisted corpus was found.
Docker Desktop	Stopped. Its only docker_data.vhdx was last modified 2026-07-15, which is before the 2026-08-17 frozen capture, so it cannot contain that snapshot. *A disk image whose last write precedes the capture date cannot hold the captured corpus. This rules the image out on its timestamp alone, without needing to mount or inspect it.*
local PostgreSQL service	None exists on the host.
matches returned	The only matches were repository schemas and reports already covered by the in-session search — no new artifact.

Conclusion. NO — the host-side search found no copy of the frozen corpus. Together with the in-session search this closes the reachable search space known to the project: the container has no copy and the owner's host has no copy.

> **WHAT THIS DOES NOT ESTABLISH**
>
> It does not prove the corpus is gone everywhere. It was not searched for on the machine that actually ran batches 001-006 if that is a different machine, nor in any off-host backup, external drive or cloud snapshot of it. The external artifact named in recovery_search.remaining_external_artifact_required is still what is required, and is still outstanding.

8. The ChatGPT-project archive lead, closed

> **EXTERNAL AUTOMATED CHATGPT-PROJECT ARCHIVE EVIDENCE — NOT PROJECT-OWNER APPROVAL**
>
> A read-only archive audit performed in a ChatGPT project and reported to this session. It approves nothing and sets no human_verified. Only the project owner may approve.
>
> Recorded 2026-08-30T11:54:02Z · lead: the August 17 archive production-rag-v1.zip in the Engineering rag system project

Provenance. Performed project-side and relayed here. It was NOT performed or independently confirmed inside this Claude container: this session cannot reach the sandbox path or hash the archive, so the audit's findings are recorded as reported, not as verified.

Conclusion. LEAD CLOSED — the archive is the early scaffold. It contains no corpus text, no populated corpus rows and no content hashes, so it does not make recovery possible. The narrowest remaining artifact is unchanged.

the archive	
path	`sandbox:/workspace/scratch/ff7d44533203/recovery/Engineering rag system/production-rag-v1.zip`
size	62,874 bytes
sha256	`99e7c13e84052ea57dcb6106636d7923f9b4f163a2f90a8f5271e5ddea99495e`
root	`production-rag-v1/`
top level	`CODE_TOOL_HANDOFF.md`, `Makefile`, `README.md`, `docker-compose.yml`, `pyproject.toml`, `data/`, `docs/`, `evals/`, `experiments/`, `scripts/`, `sql/`, `src/`, `tests/`

The audit found:

- data/raw contains only an empty placeholder
- no provider-document files
- no pg_dump or backup
- no populated corpus_snapshot_version rows
- no document_version normalized_text records
- no populated content hashes
- no real corpus manifest
- the exact snapshot ID does not appear
- sources.example.yaml has only one synthetic Widget API fixture

Constraints: no retrieval ran; no project artifact was modified.

Corroborated inside this container

the archive is far too small to contain the corpus

62,874 bytes against a corpus of at least 2,711,869 characters. Even at a generous 4x Markdown compression the corpus floor is about 678,000 bytes, roughly 11 times the archive's entire size. Scaffold-only is arithmetically consistent; a corpus could not fit.

Method: compared the reported archive size against the corpus floor implied by closed GOLD offsets, which reach character 2,711,869 across 202 documents
Note: This is an inference from the reported size, not verification of the archive itself, which this session cannot read.

the synthetic Widget fixture matches what this repository carries

It holds exactly one entry — Widget API v2, canonical_url https://example.invalid/widget/v2, local_path pointing at tests/fixtures/docs/widget_v2.md — matching the audit's description and matching the synthetic fixture found in the earlier in-session search.

Method: read data/manifests/sources.example.yaml in this checkout

THE AUGUST 17 ARCHITECTURE PACKET

It defines raw_snapshot_path only as proposed schema and contains no populated paths and no corpus rows. Not a recovery artifact. A field defined in a schema proposal carries no data; it names a place a path could have been stored, not a path.

WHAT THIS DOES NOT ESTABLISH

It closes this archive as a lead. It does not prove no other copy exists — the machine that ran batches 001-006 and any off-host backup of it remain unsearched.

9. The 2026-08-17 published results, assessed

INSUFFICIENT FOR RECOVERY

INSUFFICIENT FOR RECOVERY — the pilot remains blocked. The attachment confirms the target and strengthens the acceptance test; it supplies no corpus text and no content hashes.

Production RAG v1 — Measured Results, 2026-08-17, generated from experiments/summary.json by scripts/build_report_pdf.py · assessed 2026-08-30T11:49:13Z

What it confirms. Snapshot `snap_689e336380a054d8039dc35b2c09cd0a`; **202** documents; **14,209** chunks; and 22 evidence spans in the v1 run. 139 + 63 = 202 documents and 12,028 + 2,181 = 14,209 chunks: the figures are internally consistent.

Why it cannot recover the corpus. The document states it directly: "Raw provider documentation is not redistributed; published results carry evidence anchors, ranks and scores with retrieved text replaced by a content hash." It is a results report, not a corpus. It carries no document text at all.

reference chased	found	what it carries
`experiments/summary.json`	present, 12KB	aggregate metrics and per_case_recall only. Zero occurrences of content_hash, version_id, snap_, local_path or raw_snapshot.
`scripts/build_report_pdf.py`	present, 19KB	the report builder. Its 'retrieved text replaced by a content hash' line is a static footer stating a redaction policy — it is not a field the pipeline ever wrote, and no such hash exists in any artifact.
`per-case experiment artifacts (EXP-000 … EXP-014R)`	present	version_ids for documents that reached top-k, and ranks and scores. Zero content_hash, text_hash, chunk_hash or retrieved_text_hash fields across every artifact scanned.
`source manifests`	data/manifests/v1-openai-anthropic.yaml	202 entries with canonical_url, local_path and captured_at. No document text and no content hashes. Its local_path values point into data/raw/, which is gitignored and empty.
`document_version / content_hash records`	none	no artifact in the repository carries a document content_hash. The only file mentioning the field name is this recovery report.
`raw_snapshot_path / local_path fields`	local_path in the manifests only	paths into the empty gitignored data/raw/. raw_snapshot_path does not exist.
`corpus_snapshot_version rows`	none	no artifact carries these rows; they exist only in the PostgreSQL database, which this container does not have.

coverage measured here	
artifact files scanned	155
distinct version ids persisted	**131** of 202
distinct version ids from closed GOLD	29
document content hashes persisted	**0** of 202

131 of 202 version ids are recoverable from artifacts and 0 of 202 content hashes are. The snapshot fingerprint is a hash over (version_id, content_hash) for all 202 current versions, so it cannot be assembled from artifacts — and no artifact carries a single character of document text, so nothing can be reconstructed even in principle.

WHAT IT DOES MAKE POSSIBLE

The attachment does not recover the corpus, but it is not inert: it independently corroborates the shape a restore must have, which sharpens the acceptance test.

Implemented. provenance.FROZEN_CAPTURE_SHAPE and verify_corpus_shape() record 202 documents and 14,209 chunks as a cheap fail-closed rejection that runs before the fingerprint in scripts/rederive_unbuildable.py.

Not identity. A corpus can have the right counts and be the wrong corpus. Shape is necessary, never sufficient; the fingerprint remains the identity check and still runs.

10. The narrowest remaining artifact

Narrowed from a full pg_dump of the rag database, or the complete data/raw/ tree **to** three tables: document_version (version_id, normalized_text, content_hash, status), document_source (provider, title, canonical_url) and corpus_snapshot_version for snap_689e336380a054d8039dc35b2c09cd0a — or, equivalently, the 202 raw Markdown files at the manifest's local_path values, byte-identical to the 2026-08-17 capture.

Why this is enough. scripts/export_batch_006.py's load_docs() reads only version_id, normalized_text, provider, title, canonical_url and captured_at, and the fingerprint needs content_hash. The pilot runs no retrieval, so the chunk, embedding and search tables are not required at all.

Why nothing narrower works. The unbuildable set is defined by mining every document, so a subset of documents would produce a different set and fail the 2482 reproduction check. Partial evidence cannot substitute: 131 known version ids carry no text.

Environment prerequisite. a PostgreSQL with pgvector, which this container lacks, if restoring via the schema

11. The earlier statement of the required artifact

One of two artifacts, neither of which exists in this environment. Both must reproduce the frozen fingerprint; nothing else is admissible.

Option A — A PostgreSQL dump of the `rag` database as it stood on or after 2026-08-17.

- document_source rows for all 202 documents
- document_version rows carrying normalized_text and content_hash
- corpus_snapshot and corpus_snapshot_version rows for snap_689e336380a054d8039dc35b2c09cd0a
- *Produced by:* pg_dump of the machine that ran batches 001-006

Option B — The 202 raw Markdown files that were under data/raw/, as captured 2026-08-17T04:46:19Z.

- the files at the local_path values in data/manifests/v1-openai-anthropic.yaml
- byte-identical to the capture, since content_hash is taken over the ingested text
- *Produced by:* the gitignored data/raw/ tree on the machine that ran scripts/fetch_corpus.py
- *Note:* Re-running fetch_corpus.py today does NOT produce this: it fetches current documentation.

> **ENVIRONMENT PREREQUISITE**
> The target PostgreSQL must have the pgvector extension available. This container does not, so sql/001_init.sql cannot create the schema here even given the data.

Acceptance test. scripts/rederive_unbuildable.py refuses unless the restored corpus hashes to snap_689e336380a054d8039dc35b2c09cd0a, every closed span re-hashes at its recorded offsets, and the re-derived unbuildable count reproduces 2482. Passing all three is the proof; a snapshot row saying so is not.

12. Reproducibility safeguards added

The blocker is not only that the corpus is missing. It is that batch 006 recorded a count instead of identities, so even a restored corpus needs a re-derivation step that batch 006 made necessary. These safeguards mean a future batch does not repeat that.

Implemented in `src/rag_v1/gold/provenance.py`, tested in `tests/test_gold001_provenance.py`.

	need	provides	verified by
S1	future generation persists the unbuildable-span identities	UnbuildableLog records (version_id, char_start, char_end, evidence_text, evidence_hash, reason) for every fact a builder declines, and writes a manifest. NO_BUILDER is kept distinct from SEMANTIC_GATE, because the pilot may draw only from the former.	identity is kept not just counted; the same span twice is one entry; the two reasons stay distinguishable; the manifest carries the entries
S2	corpus snapshot fingerprint	fingerprint() reproduces the snapshot-id construction of rag_v1.snapshot.create_snapshot without a database, and verify_fingerprint() answers whether a corpus hashes to the id it claims. chunking_config_from_settings() builds the hashed config so a caller cannot silently mis-key it.	construction matches rag_v1.ids primitives exactly; one changed content hash changes the id; a missing document changes the id; a corpus claiming an id it does not hash to is rejected

	need	provides	verified by
S3	restore verifier	verify_restored_corpus() re-reads every closed span at its recorded offsets and re-hashes it against evidence_hash, reporting mismatches and missing versions rather than a bare boolean.	all 137 closed spans across batches 003-006 satisfy sha256(evidence_text) == evidence_hash; a correct restore verifies; a single changed character fails; an offset shifted by one fails; a missing document is reported, not skipped
S4	deterministic pilot-selection manifest	select_pilot_cases() orders by span identity and records a selection basis per case, so the same input returns the same ten. Semantic-gate failures and spans already spent by a closed batch are excluded; a short pool is reported short rather than padded.	selection is order-independent; takes the preregistered ten; never selects a semantic-gate failure; excludes spent spans; records why each was chosen; reports a short pool
S5	a guard so no report can call an unrun pilot passed	pilot_thresholds_unmet() returns which of the four preregistered thresholds a result fails. An absent result fails all four, because unmeasured is not met.	an unrun pilot fails all four; a passing pilot meets all four; one unsupported claim fails; seven of ten sound fails
S6	reject a wrongly-shaped restore before the expensive checks	FROZEN_CAPTURE_SHAPE and verify_corpus_shape() hold the 202 documents and 14,209 chunks the published results record, checked in the harness before the fingerprint. Chunk count is optional, since the pilot needs only document text.	the published shape is internally consistent (139+63=202, 12028+2181=14209); a correct shape passes; one document short is rejected; a wrong chunk count is rejected; shape is never treated as identity

Harness — `scripts/rederive_unbuildable.py`. Recovers the identities batch 006 discarded, by re-running batch 006's own miners and builders imported unmodified. The unbuildable set is defined by those builders as they were, so re-deriving it with a changed builder would answer a different question.

scripts/export_batch_006.py is imported, never edited. Editing it would break the very re-derivation the recovery checklist depends on.

It refuses unless:

- the corpus is reachable
- it hashes to the frozen snapshot id
- every closed span re-reads and re-hashes correctly
- the re-derived count reproduces the count batch 006 recorded

Run in this environment it refuses at the first check and writes nothing, both with PostgreSQL stopped and with it running but empty.

13. Invariants

- `retrieval_was_not_run` is still `true` and `systems_executed` is still `[]`. No retrieval system was run against any candidate at any point; SYSTEM-A and SYSTEM-B remain frozen and unexecuted.
- Closed batches modified: **0**. Dataset records modified: **0**. Eligibility state modified: **false**.
- Validation and holdout were neither inspected nor modified; no split is frozen.
- `human_verified` set by this work: **0**. Only the project owner. No AI may set human_verified, and controlled paraphrasing does not make Claude authoritative about anything.
- Files added, none modified:
  - `src/rag_v1/gold/factidentity.py` (new)
  - `src/rag_v1/gold/reasoningtype.py` (new)
  - `src/rag_v1/gold/questionscope.py` (new)
  - `tests/test_gold001_b007_fixes.py` (new)
  - `src/rag_v1/gold/provenance.py` (new)
  - `tests/test_gold001_provenance.py` (new)
  - `scripts/rederive_unbuildable.py` (new)

Next: Three independent searches have now closed: this container, the owner's host, and the ChatGPT-project archive, and the 2026-08-17 published results confirm the target without carrying corpus text. The pilot stays blocked on the narrowest remaining artifact recorded under published_results_evidence: document_version, document_source and corpus_snapshot_version for snap_689e336380a054d8039dc35b2c09cd0a, or the 202 byte-identical raw files, from the machine that ran batches 001-006 or a backup of it. With either, scripts/rederive_unbuildable.py enforces shape, fingerprint, closed-span and 2482 reproduction before any pilot case exists. The G-STRICTER finding remains open for the project owner; no AI review is owner approval.

Generated by `scripts/build_batch_007_efg_pdf.py` from the E/F/G implementation record, the batch-007 preregistration, the project-wide eligibility status and the closed batch-005 and batch-006 records. Every figure is read from those artifacts at build time. The build refuses to run if the record's state disagrees with the eligibility status, if the E/F/G checks re-run here disagree with the recorded results, if a batch-007 candidate or pilot artifact exists, or if the page would claim the pilot ran. Raw provider documentation is not redistributed; quoted spans are the short excerpts under review.